

Plataforma de Gestão de Eventos

POO II — UEG | uegPOO2

GUIA COMPLETO DE TESTES E TEXTOS DE ENTREGA

S1 até S8 — o que alterar, o que inserir e como executar tudo

Este guia acompanha o repositório **Notliw-Matos/cadastroDeEventos**. Ele explica passo a passo como rodar os testes, o que foi alterado em cada sprint e traz os textos prontos para submeter nas entregas S1 a S8 no modelo da disciplina.

PARTE 1 — GUIA DE TESTES

1. Pré-requisitos

Instale apenas:

- **JDK 11+** (java e javac no PATH). Teste: `java -version`
- **Git** para clonar o repositório.
- (Opcional para testes de banco) variáveis de ambiente `DB_URL`, `DB_USER` e `DB_PASSWORD` apontando para o Supabase da equipe.

Não é necessário Maven, Gradle, npm, PostgreSQL local nem servidor de aplicação.

2. Clonar o repositório

```
git clone https://github.com/Notliw-Matos/cadastroDeEventos.git
```

```
cd cadastroDeEventos
```

3. Estrutura de pastas relevante

```
src/main/java/domain/ - entidades e regras
src/main/java/application/ - casos de uso
src/main/java/infrastructure/ - adaptadores (banco e memória)
src/main/java/app/ - API HTTP e demos
src/test/java/ - todos os testes JUnit
scripts/ - atalhos de compilação e execução
tools/ - JUnit 5 Console Standalone (incluso)
lib/ - driver PostgreSQL (incluso)
```

4. Rodar os testes

Windows (cmd)

```
scripts\testar.bat
```

Linux / macOS / Git Bash

```
./scripts/testar.sh
```

Resultado esperado (sem banco configurado):

```
[ 11 tests skipped ] (integração com BD - ignorados sem DB_PASSWORD)
[ 153 tests successful]
[ 0 tests failed ]
```

Com banco (`DB_URL`, `DB_USER` e `DB_PASSWORD` definidos):

```
[ 0 tests skipped ]
[ 164 tests successful]
```

```
[ 0 tests failed ]
```

Os testes de integração com banco criam e apagam tudo que criam — o banco fica limpo após cada rodada.

5. Rodar a demonstração final (CA-01 a CA-07)

Windows:

```
scripts\demo-final.bat
```

Linux / macOS:

```
./scripts/demo-final.sh
```

Executa todos os cenários em memória (sem banco). Esperado: sete blocos CA-01 a CA-07, cada um com [OK] e [ERRO ESPERADO], terminando em '=== DEMONSTRACAO CONCLUIDA === todos os cenarios passaram ==='.

6. Rodar a API HTTP e abrir o site

Windows:

```
scripts\api.bat
```

Linux / macOS:

```
./scripts/api.sh
```

A API sobe em **http://127.0.0.1:8080** (use 127.0.0.1, não localhost). Abra **site/index.html** no navegador — duplo clique, sem servidor.

Sem variáveis de banco: API roda em memória com dados de demonstração já prontos (organizador@demo.com / Senha123). Com banco: dados persistem de verdade.

7. Configurar o banco Supabase (opcional)

Defina as três variáveis de ambiente — nenhuma senha vai para o código:

Windows (cmd, sessão atual):

```
set DB_URL=jdbc:postgresql://HOST:5432/postgres
set DB_USER=postgres.SEU_PROJETO
set DB_PASSWORD=sua_senha_do_banco
```

Linux / macOS / Git Bash:

```
export DB_URL=jdbc:postgresql://HOST:5432/postgres
export DB_USER=postgres.SEU_PROJETO
export DB_PASSWORD=sua_senha_do_banco
```

Eclipse: Run Configurations → aba Environment → New para cada variável.

Onde encontrar HOST e USER: painel do Supabase → Project Settings → Database → Connection string → aba Session pooler.

8. Migração do banco (primeira vez, apenas uma vez)

Abra o SQL Editor do Supabase e execute o conteúdo do arquivo Database/migracao_s4.sql. Ele adiciona as colunas `organizador_id` e `estado` em `eventos`, e `criterio_frequencia` em `atividades` — obrigatórias para RN-04 e RN-18.

O arquivo usa IF NOT EXISTS em todas as instruções: é seguro rodar de novo.

9. Carregar dados de demonstração (RNF-10)

Windows:

```
scripts\seed.bat
```

Linux / macOS:

```
./scripts/seed.sh
```

Cria 500 participantes, 1 evento publicado e 100 atividades usando os próprios casos de uso da aplicação — não por INSERT direto. Rodar de novo não duplica os participantes.

10. O que foi alterado em cada sprint

S2 — Objetos, invariantes e persistência

Criados: Email, Senha, IntervaloTempo, Vagas (objetos de valor). Atividade e Evento com invariantes. Repositórios de banco alinhados ao schema real. Corrigido: UesuarioRepositoryDatabase usava colunas inexistentes. Adicionados: EventoRepositoryDatabase e AtividadeRepositoryDatabase.

S3 — Casos de uso, portas e autenticação

Criados: CadastrarUsuarioUseCase, AutenticarUsuarioUseCase, portas de saída (UsuarioRepository, PasswordHasher), BCryptPasswordHasher. Adicionados: testes sem interface usando repositórios em memória.

S4 — Adaptadores, site público e inscrição

Criados: ServidorApi (API HTTP, 7 rotas), site/index.html, DTOs (EventoPublicoDTO, AtividadeDTO), Json.java, TokenStore. Corrigido: bug de persistência do total_inscritos (não era salvo de volta no banco). Adicionados: testes HTTP via HttpClient real (ServidorApiTest).

S5 — Políticas de inscrição e agenda

Criados: PoliticaConflitoHorario (Strategy), PoliticaConflitoBloqueante, PoliticaConflitoAlertaSomente, VerAgendaUseCase, ItemAgendaDTO, ResultadoInscricao. Rota GET /api/agenda. Site: botão 'Ver minha agenda'.

S6 — Frequência configurável

Criados: RegistroFrequenciaRepository (porta + adaptadores memória e banco), RegistrarPresencaUseCase reescrito com autorização (RN-18). Rotas: /api/frequencia/meu-codigo, /checkin, /manual, /{id}/situacao. Vendorizado: qrcode-generator (MIT) em site/vendor/. Corrigido: RegistroPresenca sempre usava LocalDateTime.now() ao reconstituir.

S7 — Avaliações, relatórios e refatoração

Criados: TipoPergunta (enum com validarResposta), Pergunta, Questionario, RespostaQuestionario (factory), CriarQuestionarioUseCase, ResponderQuestionarioUseCase, ConsultarConsolidacaoUseCase, RelatorioExportavel (interface), RelatorioInscricoes, RelatorioFrequencia, GerarRelatorioUseCase. Rotas de questionário e relatório CSV.

S8 — Integração, qualidade e entrega final

Criados: DemoFinal.java (CA-01 a CA-07 em memória), DemoFinalTest.java (11 testes JUnit cobrindo todos os cenários), README.md, DECISOES_D01_D08.md, ENDPOINTS_API.md, MODELO_DOMINIO.md, MATRIZ_RASTREABILIDADE.md, Database/migracao_s4.sql.

PARTE 2 — TEXTOS DE ENTREGA S1 A S8

Cada texto abaixo segue exatamente o modelo do arquivo s1.txt fornecido: Domínio/contexto, Glossário (quando necessário), O que foi feito, Decisão de POO, Pendências e Próxima meta. Cole o texto correspondente no campo de entrega da disciplina.

S1 — Domínio, responsabilidades e backlog

Encontros: 11/08 e 13/08/2026

Domínio:

Sistema de Cadastro de Eventos. Um Evento tem Atividades; um Usuário pode se inscrever numa Atividade (Inscrição) e ter presença registrada (Registro de Presença).

Glossário:

Evento — agrupa uma ou mais atividades.

Atividade — algo programado dentro de um evento.

Usuário — pessoa que participa.

Inscrição — vínculo entre um usuário e uma atividade.

Registro de Presença — marcação de que o usuário compareceu.

Mapa de responsabilidades:

Classes de domínio (Evento, Atividade, Usuario, Inscricao, RegistroPresenca) guardam os dados e regras básicas de cada conceito. Casos de uso (CriarEventoUseCase, RealizarInscricaoUseCase, RegistrarPresencaUseCase) executam as ações do sistema. Camada de persistência guarda os dados no banco.

Primeiro modelo justificado:

Separamos o projeto em pastas por responsabilidade (regras de negócio, casos de uso, acesso a dados) para facilitar manutenção e testes — ideia parecida com a de dividir 'o que o sistema faz' de 'como ele faz'.

Demonstração:

Rodamos o cadastro de uma atividade com sucesso (caminho válido) e testamos um caso de erro (dado inválido sendo rejeitado). Temos um teste automatizado (AtividadeTest) cobrindo esse comportamento.

Decisão de POO:

Encapsulamos a validação dentro das próprias classes de domínio (em vez de validar espalhado pelo código), garantindo que um objeto de Atividade nunca exista em estado inválido.

Pendências:

Persistir Evento e Inscrição de fato; conectar os demais casos de uso à camada de entrada.

Próxima meta:

Ter Evento e Atividade persistidos de ponta a ponta (S2).

Prazo e horário: 18/08

S2 — Objetos, invariantes e persistência

Encontros: 18/08 e 20/08/2026

Domínio:

Plataforma de Gestão de Eventos. Foco em persistir Evento e Atividade no banco e proteger suas regras dentro dos próprios objetos.

O que foi feito:

Criamos os objetos de valor Email (valida e normaliza), Senha (política mínima, nunca impressa em log), IntervaloTempo (fim > início obrigatório) e Vagas (limitada ou ilimitada). Evento e Atividade agora protegem suas invariantes: Evento só muda de estado via publicar()/encerrar(), e Atividade só ganha inscrito via incrementarInscrito(), que lança VagasEsgotadasException se não houver vaga. Adaptadores de banco (EventoRepositoryDatabase, AtividadeRepositoryDatabase) foram alinhados ao schema real do Supabase.

Demonstração:

Caminho válido: criar Evento em RASCUNHO e publicar — o estado muda corretamente. Erro relevante: tentar criar IntervaloTempo com fim anterior ao início — rejeitado na construção. Teste: AtividadeTest.invarianteDeVagasNuncaEULtrapassada e IntervaloTempoTest.

Decisão de POO:

Invariante protegida: Vagas e IntervaloTempo validam na construção (objeto de valor — ROO-03). Atividade só ganha inscrito pelo método incrementarInscrito(); não existe caminho para criar objeto inválido (encapsulamento — ROO-02). Alternativa descartada: validar nos casos de uso — significa que o mesmo objeto pode existir em estado inválido se o caso de uso for esquecido.

Refatoração:

Antes: UsuarioRepositoryDatabase usava colunas inexistentes no banco real (atualizado_em, perfil_usuario). Depois: SQL alinhado ao schema confirmado — sem migration, só correção.

Pendências:

Casos de uso de cadastro e login ainda sem autenticação de verdade (hash de senha). Repositório de Inscrição sem adaptador de banco.

Próxima meta:

Cadastro, login com bcrypt e portas de entrada/saída (S3).

Prazo e horário: 25/08

S3 — Casos de uso, portas e autenticação

Encontros: 25/08 e 27/08/2026

Domínio:

Plataforma de Gestão de Eventos. Foco em cadastro, login e autorização, com dependências invertidas e testes sem interface.

O que foi feito:

Criamos `CadastrarUsuarioUseCase` (RF-01, RN-01) e `AutenticarUsuarioUseCase` (RF-02) como casos de uso com portas de saída injetadas: `UsuarioRepository` e `PasswordHasher` (interfaces do domínio). `BCryptPasswordHasher` implementa o hash compatível com os hashes do seed (`crypt(..., gen_salt('bf'))`). Perfil PARTICIPANTE é o único gerado por cadastro público — nunca pelo formulário. Autorização no servidor: `CriarEventoUseCase` verifica a permissão no caso de uso, não na tela (RNF-06).

Demonstração:

Caminho válido: cadastrar participante → login retorna token → organizador cria evento. Erro relevante: e-mail duplicado (mensagem clara, RN-01); senha errada (mesma mensagem de e-mail inexistente — não revela quais e-mails existem); participante tentando criar evento (`AcessoNegadoException`). Teste sem interface: `CadastrarUsuarioUseCaseTest` e `AutenticarUsuarioUseCaseTest` usam repositório em memória — sem banco, sem HTTP.

Decisão de POO:

Inversão de dependência (ROO-08/09): os casos de uso dependem de `UsuarioRepository` e `PasswordHasher` (interfaces). Trocar `bcrypt` por outro algoritmo ou `Postgres` por memória não toca em nenhuma regra de negócio — só o adaptador muda. Alternativa descartada: chamar o banco direto no caso de uso — impossibilitaria testar sem banco.

Pendências:

Adaptadores de banco para `Inscricao`. Sem token de sessão persistido — o login devolve o usuário mas não há middleware de autenticação na API (S4).

Próxima meta:

API HTTP, site público e inscrição integrada (S4).

Prazo e horário: 01/09

S4 — Adaptadores, site público e inscrição

Encontros: 01/09 e 03/09/2026

Domínio:

Plataforma de Gestão de Eventos. Foco em expor os casos de uso via API HTTP, consumida pelo site público e por qualquer cliente futuro.

O que foi feito:

Criamos ServidorApi usando apenas com.sun.net.httpserver.HttpServer (JDK, sem framework) e um parser JSON de 100 linhas escrito à mão. Rotas: cadastro, login, listar eventos publicados, programação, criar evento/atividade e inscrição. TokenStore emite tokens opacos (UUID) sem senha embutida. Site público: site/index.html com fetch() — sem build, sem framework, abre com duplo clique. Corrigido bug real: RealizarInscricaoUseCase não chamava atividadeRepository.salvar() após incrementarInscrito(), então a vaga nunca era persistida no banco. Encontrado só quando os testes de integração rodaram contra Postgres de verdade (RNF-12 em ação).

Demonstração:

Caminho válido: organizador loga → cria evento → cria atividade → site lista o evento. Erro relevante: POST /api/eventos sem token (401); participante com token (403, RNF-06). Teste HTTP: ServidorApiTest sobe HttpServer real numa porta livre e usa HttpClient — sem curl, sem mock.

Decisão de POO:

Separação DTO/domínio (ROO-09): EventoPublicoDTO nunca expõe organizadorId. Composition root: ServidorApi.criar() é o único lugar que conhece os adaptadores concretos — domínio e casos de uso não sabem que existe HTTP. Alternativa descartada: Spring Boot (40 MB de dependências, oculta o roteamento, dificulta explicação na prova).

Pendências:

Schema real de eventos/atividades confirmado em 30/09: migração de organizador_id e estado necessária (ver Database/migracao_s4.sql). Repositórios de Questionario e RespostaQuestionario só em memória (S7). Token sem expiração.

Próxima meta:

Políticas de inscrição e agenda pessoal (S5).

Prazo e horário: 08/09

S5 — Políticas de inscrição e agenda

Encontros: 08/09 e 10/09/2026

Domínio:

Plataforma de Gestão de Eventos. Foco em resolver o conflito de horário por composição e polimorfismo, e expor a agenda pessoal do participante.

O que foi feito:

Criamos a interface PoliticaConflitoHorario com dois implementadores: PoliticaConflitoBloqueante (padrão — lança exceção) e PoliticaConflitoAlertaSomente (deixa inscrever, mas devolve avisos). RealizarInscricaoUseCase recebe a política pelo construtor e nunca sabe qual está ativa (polimorfismo — ROO-05). VerAgendaUseCase: retorna inscrições ativas em ordem cronológica (RF-17, RN-08). Rota GET /api/agenda (autenticado). Site: botão 'Ver minha agenda'. POST

/api/inscricoes agora devolve avisos no JSON.

Demonstração:

Caminho válido: participante se inscreve em duas atividades sem conflito → agenda mostra as duas em ordem. Erro relevante: segunda inscrição conflitante com política bloqueante → `ConflitoDeHorarioException`. Teste: `comPoliticaDeAlertaAmbasAsInscricoesSaoAceitasEGeraAviso` — duas inscrições com sobreposição de horário, aviso no retorno.

Decisão de POO:

Strategy (ROO-05/10): RN-07 diz 'alertar OU bloquear por uma política uniforme'. Ter a política como objeto injetado significa que trocar de comportamento é trocar um construtor, sem mexer no caso de uso. Alternativa descartada: flag booleana — cada nova regra viraria mais um if.

Refatoração documentada:

Antes: o caso de uso calculava sobreposição internamente com lógica de data dispersa. Depois: `Atividade.conflitaCom()` + `IntervaloTempo.temConflitoCom()` — a regra mora no objeto que a conhece. Teste `IntervaloTempoTest` cobre diretamente.

Pendências:

Cancelamento de inscrição (RF-15) não tem rota exposta na API.

Próxima meta:

Frequência configurável, QR Code e lançamento manual (S6).

Prazo e horário: 15/09

S6 — Frequência configurável

Encontros: 15/09 e 17/09/2026

Domínio:

Plataforma de Gestão de Eventos. Foco em registrar presença via QR Code e lançamento manual, com critério configurável por atividade.

O que foi feito:

Criamos `RegistroFrequenciaRepository` (porta de saída) com adaptadores em memória e banco. `RegistrarPresencaUseCase` reescrito: resolve o critério de frequência da própria atividade (ROO-05 — não precisa que o chamador passe a estratégia) e verifica autorização do mesmo jeito que `CriarAtividadeUseCase` (RN-18). Rotas: GET `/api/frequencia/meu-codigo` (gera código opaco — RN-10), POST `/api/frequencia/checkin`, POST `/api/frequencia/manual`, GET `/api/frequencia/{id}/situacao`. Site: seções de QR, check-in e lançamento manual. QR gerado com `qrcode-generator` (MIT, vendorizado, testei que gera SVG real com Node antes de confiar).

Demonstração:

Caminho válido: Ana gera QR → organizador faz check-in → situação = PRESENTE (CHECK_IN_UNICO). Oficina com ENTRADA_SAIDA: só check-in → parcial; check-in + check-out

→ PRESENTE. Lançamento manual: organizador registra sem QR, operador fica gravado (RN-11). Erro relevante: outro organizador tenta registrar em evento alheio → `AcessoNegadoException` (RN-18). Teste: `RegistrarPresencaUseCaseTest` (3 critérios + 3 testes de autorização).

Decisão de POO:

`CriterioFrequencia` (enum com Strategy embutido — ROO-05): cada constante tem seu `ValidadorFrequenciaStrategy`. O caso de uso chama `atividade.getCriterioFrequencia().validador().isPresencaValida(registros)` — sem nenhum if por critério. Alternativa descartada: `switch(criterio)` no caso de uso — ponto de mudança obrigatório para cada novo critério.

Bug encontrado:

`RegistroPresenca` sempre usava `LocalDateTime.now()` — inclusive ao reconstituir do banco. Registros antigos pareceriam ter acabado de acontecer. Corrigido com segundo construtor que aceita `dataHora` explícita. Pego pelo teste `PersistenciaIntegracaoTest.registroDeFrequenciaPersisteComDataHoraRealEAutorizacao`.

Pendências:

Código QR sem expiração (TTL). Leitura por câmera não implementada — organizador digita/cola o código.

Próxima meta:

Avaliações, relatórios e refatoração documentada (S7).

Prazo e horário: 22/09

S7 — Avaliações, relatórios e refatoração

Encontros: 22/09 e 24/09/2026

Domínio:

Plataforma de Gestão de Eventos. Foco em avaliações configuráveis, relatórios exportáveis em CSV e a refatoração mais significativa do projeto.

O que foi feito:

Avaliações: `TipoPergunta` (enum com `validarResposta` — ROO-05), `Pergunta`, `Questionario`, `RespostaQuestionario.criar()` (factory que valida tudo antes de criar — ROO-02). Casos de uso: `CriarQuestionarioUseCase` (RF-24), `ResponderQuestionarioUseCase` (RF-26, RF-27, RN-13, RN-14), `ConsultarConsolidacaoUseCase` (RF-28). Relatórios: interface `RelatorioExportavel` com dois implementadores (`RelatorioInscricoes` e `RelatorioFrequencia`). `GerarRelatorioUseCase` devolve o objeto correto; a rota só chama `toCsv()` — não sabe qual dos dois recebeu. Rotas de questionário e CSV com Content-Disposition: attachment.

Demonstração:

Caminho válido: criar questionário com 3 tipos de pergunta → Ana (presente) responde → consolidação mostra média, distribuição e comentários. Erro relevante: participante sem presença não avalia (RN-13); Ana tenta responder duas vezes (RN-14). Relatório: GET /api/atividades/{id}/relatorio/inscricoes retorna CSV abrível fora do sistema (RF-31). Teste: AvaliacaoTest (11 testes), ResponderQuestionarioUseCaseTest, GerarRelatorioUseCaseTest.

Decisão de POO:

TipoPergunta (Strategy no enum — ROO-10): acrescentar um quarto tipo é acrescentar uma constante — nenhum if em Pergunta, nos casos de uso ou nas rotas precisa mudar. Alternativa descartada: hierarquia de subclasses (PerguntaTexto, PerguntaEscala, etc.) — comportamento diferente é só a validação, não justifica hierarquia aberta.

Refatoração documentada (antes/depois):

Antes: lógica de formatação CSV ficaria inline nas rotas (acoplamento HTTP + formato). Depois: RelatorioExportavel (interface) + implementadores. A rota só chama toCsv() e nomeArquivo(). Efeito: adicionar PDF no S8 é criar uma nova classe — rota, caso de uso e domínio não mudam (Open/Closed — ROO-08).

Pendências:

Repositórios de Questionario e RespostaQuestionario só em memória — avaliações somem ao reiniciar.

Próxima meta:

Integração final, CA-01 a CA-07, ROO rastreados e documentação (S8).

Prazo e horário: 29/09

S8 — Integração, qualidade e entrega final

Encontros: 29/09 e 01/10/2026

Domínio:

Plataforma de Gestão de Eventos. Entrega final com todos os cenários CA-01 a CA-07 integrados, ROO-01 a ROO-12 rastreados e documentação completa.

O que foi feito:

DemoFinal.java: executa CA-01 a CA-07 em memória, sem banco, sem interface — cada cenário mostra caminho válido e erro relevante. DemoFinalTest.java: os mesmos cenários como testes JUnit (11 testes novos). Documentação entregue: README.md (pré-requisitos, configuração, execução — RNF-15), DECISOES_D01_D08.md, ENDPOINTS_API.md, MODELO_DOMINIO.md (diagramas Mermaid), MATRIZ_RASTREABILIDADE.md (RF-01 a RF-36 × ROO). Total: 164 testes passando com banco / 153 sem banco.

Demonstração:

Rodar scripts/demo-final.bat (ou .sh): sete blocos CA-01 a CA-07, todos [OK] e [ERRO ESPERADO] corretos, terminando em 'DEMONSTRACAO CONCLUIDA'. Rodar scripts/testar.bat: 153 testes sem banco, 164 com banco. Rodar scripts/api.bat + abrir site/index.html: fluxo completo de inscrição e frequência no browser.

Decisão de POO (síntese):

Os três pontos de variação com Strategy (PoliticaConflitoHorario, CriterioFrequencia, TipoPergunta) são os mais fáceis de explicar na prova: em cada um, acrescentar um novo comportamento é acrescentar uma constante ou uma classe nova — sem if nos chamadores. RelatorioExportavel demonstra Open/Closed: a rota nunca precisa ser modificada para novos formatos.

Limitações declaradas:

RF-08 (vincular palestrantes) não implementado. RF-15 (cancelamento pela API) não exposto. RF-32 a RF-35 (certificados, Desejável D) não implementados. Repositórios de avaliação só em memória. Token QR sem expiração.

Pendências:

Nenhuma obrigação pendente. Extensões futuras documentadas em S8_ENTREGA.md.

Próxima meta:

Entrega final concluída.

Prazo e horário: 01/10

Resumo dos scripts disponíveis

Script	O que faz	Precisa de banco?
scripts/testar	Compila e roda todos os testes JUnit	Não (ignora os 11 de BD)
scripts/demo-final	CA-01 a CA-07 em memória	Não
scripts/api	Sobe a API em http://127.0.0.1:8080	Opcional
scripts/seed	500 participantes + 100 atividades no banco	Sim
scripts/demo	Demo rápida S2/S3 contra o banco	Sim

Repositório: github.com/Notliw-Matos/cadastroDeEventos | POO II — UEG